Practical - 08

Aim: To Study about Lexical Analyzer Generator (LEX) and Flex (Fast Lexical Analyzer).

Lex - A Lexical Analyzer Generator:-

Lex is a program generator designed for lexical processing of character input streams. It accepts a high-level, problem oriented specification for character string matching, and produces a program in a general purpose language which recognizes regular expressions. The regular expressions are specified by the user in the source specifications given to Lex. The Lex written code recognizes these expressions in an input stream and partitions the input stream into strings matching the expressions. The grammar in the above diagram is a text file you create with a text edtior. Yacc will read your grammar and generate C code for a syntax analyzer or parser. The syntax analyzer uses grammar rules that allow it to analyze tokens from the lexical analyzer and create a syntax tree. The syntax tree imposes a hierarchical structure the tokens. For example, operator precedence and associativity are apparent in the syntax tree. The next step, code generation, does a depth-first Lexical Analyzer Syntax Analyzer a = b + c * d id1 = id2 + id3 * id4 = + * id1 source code tokens syntax tree id2 id3 id4 load id3 mul id4 add id2 store id1 Code Generator generated code Lex Yacc patterns grammar 5 walk of the syntax tree to generate code. Some compilers produce machine code, while others, as shown above, output assembly language.

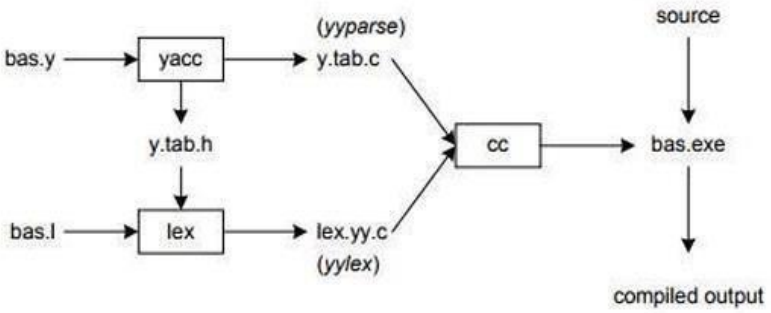


Figure 2: Building a Compiler with Lex/Yacc

What is Flex?

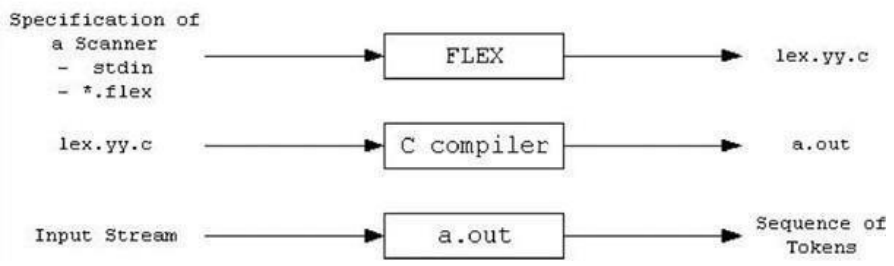
• Flex is a powerful, open source application framework which allows to build traditional applications for browser, mobile and desktop using the same programming model, tool, and codebase.

• Flex provides FLEX SDK consisting of the Flex class library (ActionScript classes), the Flex compilers, the debugger, the MXML and ActionScript programming languages, and other utilities to build expressive and interactive rich internet applications (RIA)

• Flex takes care of the user interface (UI) or the client-side functionality of a web application. Server-side functionality dependent on server-side components written in a traditional scripting language

How to use FLEX?

FLEX (Fast LEXical analyzer generator) is a tool for generating scanners. In stead of writing a scanner from scratch, you only need to identify the vocabulary of a certain language (e.g. Simple), write a specification of patterns using regular expressions (e.g. DIGIT [0-9]), and FLEX will construct a scanner for you. FLEX is generally used in the manner depicted here:



First, FLEX reads a specification of a scanner either from an input file *.lex, or from standard input, and it generates as output a C source file lex.yy.c. Then, lex.yy.c is compiled and linked with the "-lfl" library to produce an executable a.out. Finally, a.out analyzes its input stream and transforms it into a sequence of tokens.

*.lex is in the form of pairs of regular expressions and C code. (sample1.lex, sample2.lex) lex.yy.c defines a routine yylex() that uses the specification to recognize tokens. a.out is actually the scanner!

How to Compile & Run LEX / YACC Programs on Windows ?

If you are installing Ubuntu (or any Linux based OS) on your system either through Virtual Box or by making your system Multi-Bootable, just to execute your Lex & Yacc programs; then you might be wasting your HDD space & your valuable time. You can easily skip this annoying process and run your programs in Windows OS without any hassles.

Here's how you can do it:

Installing Softwares:

1. Download Flex 2.5.4a
2. Download Bison 2.4.1
3. Download DevC++
4. Install Flex at "C:\GnuWin32"
5. Install Bison at "C:\GnuWin32"
6. Install DevC++ at "C:\Dev-Cpp"
7. Open Environment Variables.
8. Add "C:\GnuWin32\bin;C:\Dev-Cpp\bin;" to path.

Compilation & Execution of your Program:

1. Open Command prompt and switch to your working directory where you have stored your lex file (".l") and yacc file (".y")
2. Let your lex and yacc files be "hello.l" and "hello.y". Now, follow the preceding steps to compile and run your program.
    1. For Compiling Lex file only:
        1. flex hello.l
        2. gcc lex.yy.c
    2. For Compiling Lex & Yacc file both:
        1. flex hello.l
        2. bison -dy hello.y
        3. gcc lex.yy.c y.tab.c
3. For Executing the Program
    1. a.exe

EXAMPLE:- **HELLO.L FILE**

```
%{
#include "y.tab.h"

int yyerror(char *errormsg);
%}
```

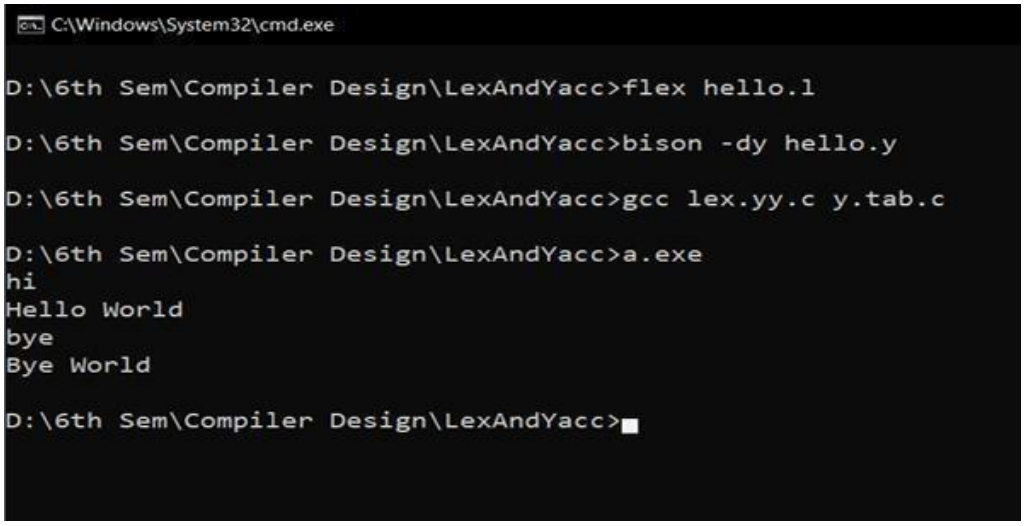
```
%%

("hi"|"oi")"\n"     { return HI; }
("tchau"|"bye")"\n" { return BYE; }
.                   { yyerror("Unknown char"); }

%%
int main(void)
{
    yyparse();
    return 0;
}
int yywrap(void)
{
    return 1;
}
int yyerror(char *errormsg)
{
    fprintf(stderr, "%s\n", errormsg);
    exit(1);
}
```

HELLO.Y FILE

```
%{
#include <stdio.h>
#include <stdlib.h>

int yylex(void);
int yyerror(const char *s);
%}
```

```
%token HI BYE

%%
program: hi bye ;

hi: HI { printf("Hello World\n"); } ;

bye: BYE { printf("Bye World\n"); exit(0); } ;
%%
int yyerror(const char *s)
{
    fprintf(stderr, "Error: %s\n", s);
    return 1;
}
int main(void)
{
    return yyparse();
}
```

OUTPUT:



Practical - 09

Aim: A) Create a Lexer to take input from text file and count no of characters, no. of lines & no. of words.

Program :

```
%{
#include <stdio.h>
int lines = 0, words = 0, s_letters = 0, c_letters = 0, num = 0, spl_char = 0, total = 0;
%}

%%
\n        { lines++; words++; }
[ \t]     { words++; }
[A-Z]     { c_letters++; }
[a-z]     { s_letters++; }
[0-9]     { num++; }
.         { spl_char++; }
%%

int main(void)
{
    FILE *fp = fopen("practical9.txt", "r");
    if (!fp)
    {
        perror("Error opening file");
        return 1;
    }
    yyin = fp;
    yylex();
```

```
 fclose(fp);

    total = s_letters + c_letters + num + spl_char;

    printf("This file contains ...");
    printf("\n\t%d lines", lines);
    printf("\n\t%d words", words);
    printf("\n\t%d small letters", s_letters);
    printf("\n\t%d capital letters", c_letters);
    printf("\n\t%d digits", num);
    printf("\n\t%d special characters", spl_char);
    printf("\n\tIn total %d characters.\n", total);

    return 0;
}

int yywrap()
{
    return 1;
}
```

Practical9.txt FILE :

```
Hii
I'm Karthik From CSE.
```

OUTPUT :

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19042.804]
(c) 2020 Microsoft Corporation. All rights reserved.

D:\6th Sem\Compiler Design\LexAndYacc>flex practical9.l

D:\6th Sem\Compiler Design\LexAndYacc>gcc lex.yy.c

D:\6th Sem\Compiler Design\LexAndYacc>a.exe
 This File contains ...
        1 lines
        7 words
        12 small letters
        5 capital letters
        0 digits
        0 special characters
        In total 17 characters.

D:\6th Sem\Compiler Design\LexAndYacc>
```

B). Write a Lex program to count number of vowels and consonants in a given input string.

Program :

```
%{
#include   <stdio.h>
int  vow_count  =  0;
int const_count = 0;
%}
%%
[aeiouAEIOU] { vow_count++; }
[a-zA-Z] { const_count++; }
[^aeiouAEIOU \n\t] { const_count++; }

%%

int main()
{
```

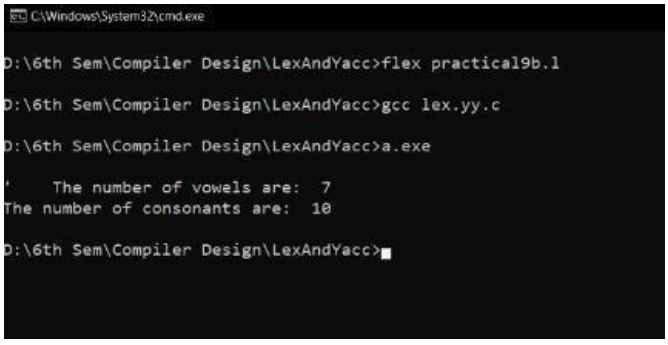
```
    FILE *fp = fopen("practical8.txt", "r");
        if (!fp)
        {
            perror("Error opening file");
            return 1;
        }
        yyin = fp;
        yylex();
        fclose(fp);
printf("The number of vowels are: %d\n", vow_count);
printf("The number of consonants are: %d\n", const_count);

        return 0;
    }
    int yywrap()
    {
        return 1;
    }
```

practical8.txt FILE

Hii

I'm Karthik From CSE

OUTPUT:



Practical - 10

Aim:- A). Write a Lex program to print out all numbers from the given file.

Program :

```
%{
#include <stdio.h>
int num = 0;
%}
%%

[0-9] { num++; ECHO; }

%%
int main(void)
{
    FILE *fp = fopen("practical10.txt", "r");
    if (!fp)
    {
        perror("Error opening file");
        return 1;
    }
    yyin = fp;
    yylex();
    fclose(fp);

    printf("\n\t%d digits\n", num);

    return 0;
}
```

```
int yywrap()
{
    return 1;
}
```

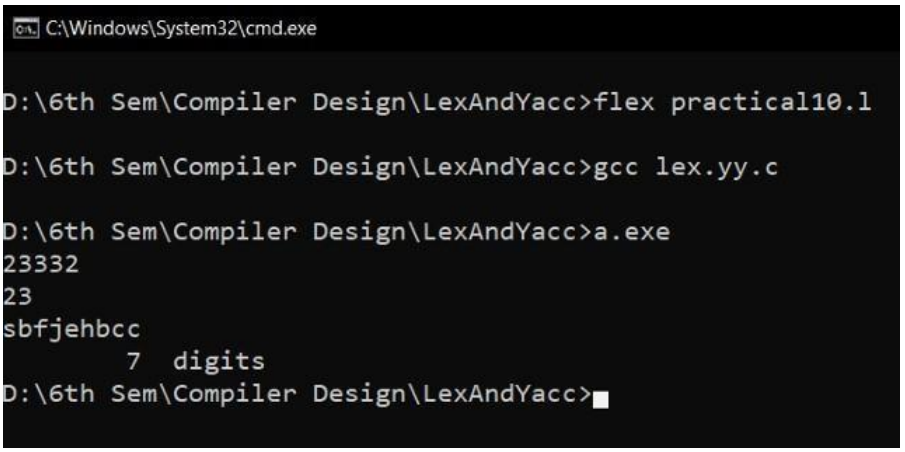
Practical10.txt FILE

23332

23

Sbfjehbcc

OUTPUT:



B). Write a Lex program to printout all HTML tags in file.

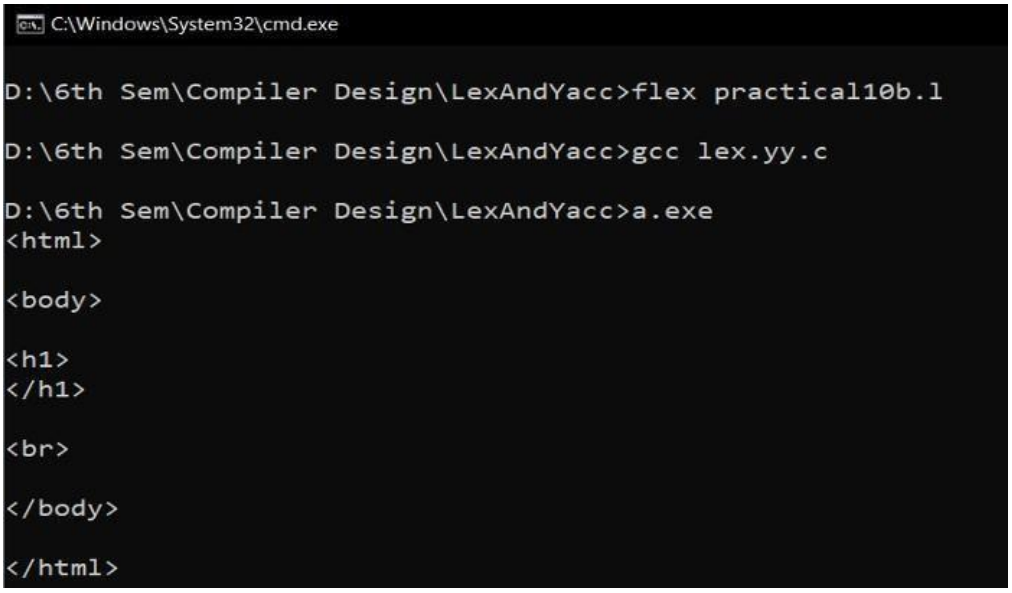
Program :

```
%{
#include <stdio.h>
%}
%%
\<[^>]*\> { printf("%s\n", yytext); }
.|\n     { /* Ignore other characters */ }

%%
int yywrap()
{
    return 1;
}
int main()
{
    FILE *fp = fopen("practical10b.txt", "r");
    if (!fp)
    {
        perror("Error opening file");
        return 1;
    }
    yyin = fp;
    yylex();
    fclose(fp);

    return 0;
}
```

Practical10b.txt FILE

```
<html>
<body>
<h1> Hello</h1>
<br>
</body>
</html>
```

OUTPUT:



C). Write a Lex program which adds line numbers to the given file and display the same onto the standard output.

Program :

```
%{
#include <stdio.h>

int line_number = 1;
%}
```

```
%%

.*\n { printf("%d %s", line_number++, yytext); }

%%

int yywrap()
{
    return 1;
}

int main()
{
    FILE *fp = fopen("add.txt", "r");
    if (!fp)
    {
        perror("Error opening file");
        return 1;
    }

    yyin = fp;
    yylex();
    fclose(fp);

    return 0;
}
```

Add.txt FILE

```
#include <stdio.h>
int main()
{
    int a = 10, b = 20;
    int c = a + b;

    printf("Addition of a and b is: %d\n", c);

    return 0;
}
```

OUTPUT:

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Version 10.0.19042.804]
(c) 2020 Microsoft Corporation. All rights reserved.

D:\6th Sem\Compiler Design\All Programs\2>flex AddLineNumber.l

D:\6th Sem\Compiler Design\All Programs\2>gcc lex.yy.c

D:\6th Sem\Compiler Design\All Programs\2>a.exe
1 #include<stdio.h>
2 int main()
3 {
4 int a=10,b=20;
5 int c=a+b;
6 printf("Addition of a and b is:%d",c);
7 return 0;
8 }

D:\6th Sem\Compiler Design\All Programs\2>
```